

JUSTIFICACIÓN PRINCIPIOS SOLID

EJERCICIO
Aseguradora

En el diseño de nuestro sistema para la aseguradora, aplicamos los 5 principios SOLID para garantizar que el código sea fácil de mantener y no se rompa si necesitamos agregar más cosas en el futuro. A continuación, explicamos cómo se aplicó cada uno:

1. S - Single Responsibility Principle (Principio de responsabilidad única)

- **Aplicación en el código:** Este principio se refleja en la clara separación de responsabilidades a través de distintas clases. La clase *Datos* tiene el único propósito de interactuar con el usuario para capturar y validar información (mediante métodos como *ingresar_edad*). Por su parte, la clase *ConversorMoneda* se dedica exclusivamente a transformar divisas con su método *mxn_a_usd*. Gracias a esto, la clase principal *CalculadoraSeguro* no requiere manejar entradas ni conversiones externas, limitándose a calcular la prima matemática.

2. O - Open/Closed Principle (Principio de abierto/cerrado)

- **Aplicación en el código:** Se implementó utilizando el patrón de diseño *Strategy* mediante la clase abstracta *FactorEdadStrategy*. Esta clase base permite que el sistema esté "abierto" a la creación de nuevas categorías en el futuro. De la misma forma, el método *calcular_prima* dentro de *CalculadoraSeguro* permanece "cerrado" a modificaciones, ya que su comportamiento se adapta sin necesidad de alterar el código fuente existente ante nuevas cosas.

3. L - Liskov Substitution Principle (Principio de sustitución de Liskov)

- **Aplicación en el código:** Las clases *FactorFemenino* y *FactorMasculino* heredan de *FactorEdadStrategy* y cumplen estrictamente con su contrato al implementar el método *obtener_factor(edad_ajustada)*. Esto permite que, en el método *calcular_prima*, la variable *estrategia_factor* pueda almacenar cualquiera de las dos instancias y ejecutar el cálculo sin que el sistema presente errores de compatibilidad.

4. I – Interface Segregation Principle (Principio de segregación de interfaces)

- **Aplicación en el código:** La interfaz definida por la clase abstracta *FactorEdadStrategy* es precisa y concisa; únicamente exige la implementación del método *obtener_factor*. Esto evita que las clases *FactorFemenino* y *FactorMasculino* tengan que cargar con métodos innecesarios que no corresponden a su naturaleza.

5. D – Dependency Inversion Principle (Principio de inversión de dependencias)

- **Aplicación en el código:** Este principio se aplica en el constructor (*__init__*) de la clase *CalculadoraSeguro*. En lugar de crear y configurar rígidamente la conversión de moneda dentro de la calculadora, esta recibe el objeto *conversor* (una instancia de *ConversorMoneda*) desde el exterior mediante inyección de dependencias. Si en el futuro la empresa decide cambiar la clase del conversor (por ejemplo, para consultar la tasa en tiempo real vía internet), la clase *CalculadoraSeguro* seguirá funcionando sin requerir ninguna modificación en su código interno.